

计网复习

第一部分

本PPT只是概述！
本PPT可能存在错误！

如果本PPT出现和教材/PPT/老师所讲的知识存在
出入，那么请以教材/PPT/老师所讲的知识为准！

Computer Networks and the Internet

First, we can describe the nuts and bolts of the Internet, that is, the basic hardware and software components that make up the Internet. Second, we can describe the Internet in terms of a networking infrastructure that provides services to distributed applications.

Basic Concepts of Internet

- **Internet 互联网**: 以**TCP/IP协议为主**的一簇协议, 由该协议支撑起计算机网络。
- **Computer Network 计算机网络**是一个将**分散的**(地理位置不同的)、具有独立功能的**计算机系统**, 通过**通信设备**(路由等)与线路(光纤等)连接起来, 由功能完善的软件实现资源共享和信息传递的系统。
- 简而言之: 计算机网络是**互连的、自治的计算机集合**.
 - 互连: 通过通信链路互联互通
 - 自治: 无主从关系
- **Link 链路**: 链路的作用是将各个节点连接在一起。其可分为接入网链路和主干链路, 前者指主机连接到互联网的链路; 后者指路由器之间的链路。
- **Node 节点**: 节点可分为两类: 主机节点和数据交换节点。前者可以是主机及其上运行的应用程序(是数据的源或目标); 后者可以是路由器、交换机等网络交换设备(既不是数据的源也不是目标, 而是数据的中转节点)。

Basic Concepts of Internet

- **Protocol 协议**: 定义了在两个或多个通信实体之间交换的报文的格式和顺序, 以及报文发送和/或接收一条报文或其他事件所采取的动作。在互联网中, 所有的通信协议都由协议制约。协议规定了通信实体之间所交换的消息的格式、意义、顺序以及针对收到信息或发生的事件所采取的动作。协议有三要素: 语法(数据与控制信息的结构或格式、信号电平)、语义(需要发出何种控制信息、完成何种动作以及做出何种响应、差错控制)、时序(事件顺序、速度匹配)。
- **Service 服务**: 在协议的控制下, 本层向上一层提供服务, 本层使用下一层所提供的服务。
- **Transmission Rate 速率 (数据传输速率、数据率、比特率)**: 指连接到计算机网络上的主机在数字信道上传送数据位数的速率。其单位为b/s (比特/秒, bit/s, 有时也写为bps)。最高数据传输速率即为带宽。
 - 在传输领域中 (通信), 传输速率采用的是十进制, $K=10^3$
 - 在存储领域中, 存储容量等使用的是二进制, $K=2^{10}$
- **Bandwidth 带宽**: 表示网络的通信线路所能传送数据的能力。通常是指单位时间内从网络中的某一点到另一点所能通过的最高数据传输速率。其单位为b/s。

Network Structure: Core and Edge

Network Core 网络核心：大量的网络和连接网络的路由器，为边缘部分提供连通性和交换服务

- 路由-决定分组采用的源主机到目标主机的路径(全局)
- 转发-将分组从路由器的输入链路转移到输出链路(局部)

Network Edge 网络边缘：所有连接到Internet上、可供用户直接使用的端系统

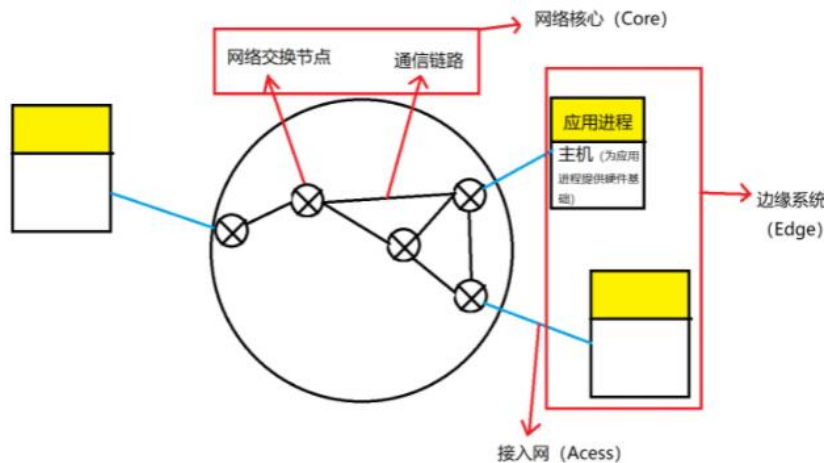
- 端系统(主机和服务器)

Access Networks 接入网：将端系统与网络核心相连接

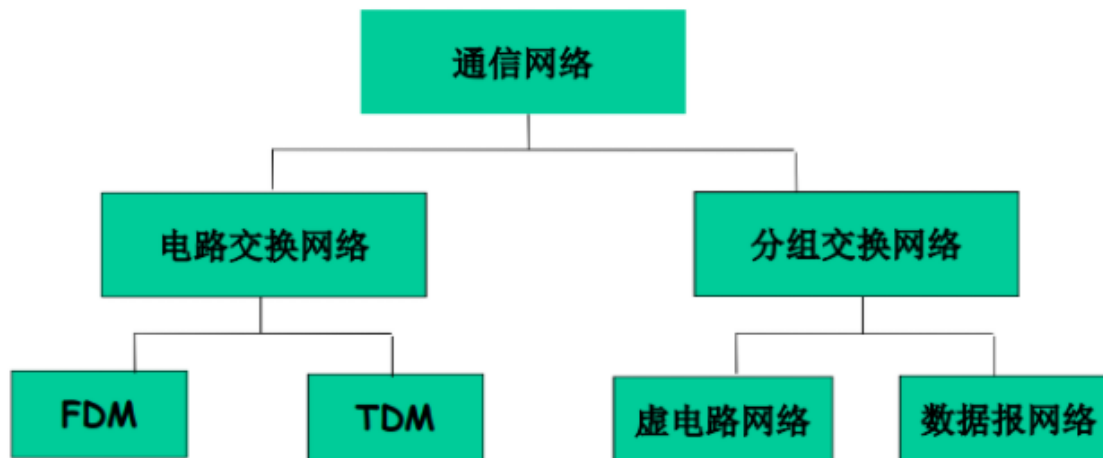
- DSL - Digital Subscriber Line
- ADSL - Asymmetric Digital Subscriber Line
- HFC - Hybrid Fiber-Coaxial
- FDDI - Fiber Distributed Data Interface

Physical Media 传输介质：

- Bit 在发送-接收者对之间传送的内容
- Physical link 发送-接收者对之间的物理链路
- Guided media 引导型媒体：在固体媒介中传播的信号，如铜线、光纤、同轴线缆
- Unguided media 非引导型媒体：自由传播/无线传播的信号，如无线电



Network Core : Circuit-Switching/Packet-Switching



The Classification of Communication Network

Network Core : Circuit-Switching

电路交换

Circuit switching: 端到端的资源被分配给从源端到目标端的呼叫（建立起一条专有链路）。

例子：如右图，每段链路有4条链路。从左上到右下角主机间建立了呼叫。该呼叫采用了上面链路的第2个线路，右边链路的第1个线路。

电路交换网络的特点是**资源独享**，即便主机间建立连接后无数据传输，也会占用线路资源，从而使其他用户无法使用，导致资源浪费。但是这种连接方式保证了性能，**多用于传统电话网络，而不适用计算机间的连接**

为了提高电路交换网络的使用效率，网络资源（如带宽）被分成“片段”，同时被多位用户使用。像这样分成片段的方式主要有：

- 频分 (Frequency-division multiplexing)
- 时分 (Time-division multiplexing)
- 波分 (Wavelength-division multiplexing)

FDM

TDM

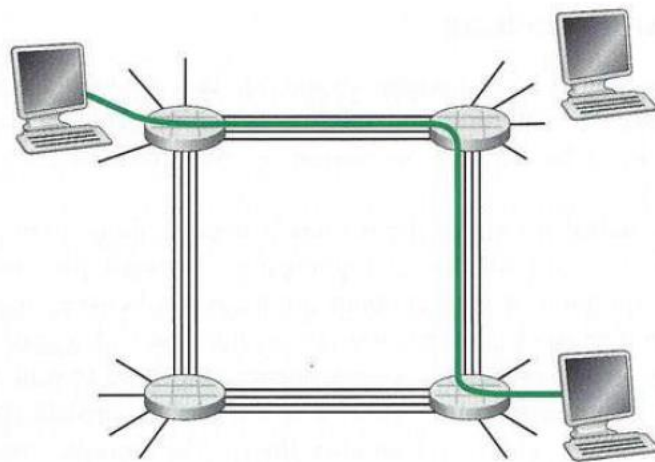


Figure 1.13 ♦ A simple circuit-switched network consisting of four switches and four links

Network Core: Circuit-Switching: FDM/TDM

FDM-Frequency Division Multiplexing 频分多路复用:

- 将带宽分成几个片段, 不同的用户使用不同的片段。

划为频率

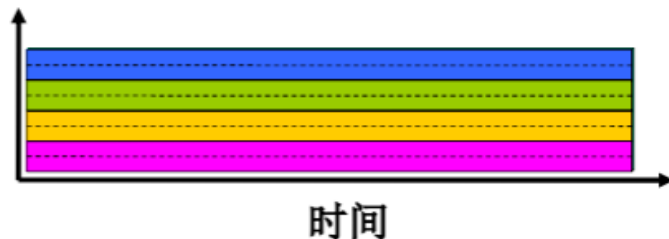
TDM-Time Division Multiplexing 时分多路复用

- 将带宽按周期分为成许多片段, 每一个周期再分为几个不同的时隙 (slot), 每位用户固定使用其中的一个时隙。

划为时间

FDM

频率



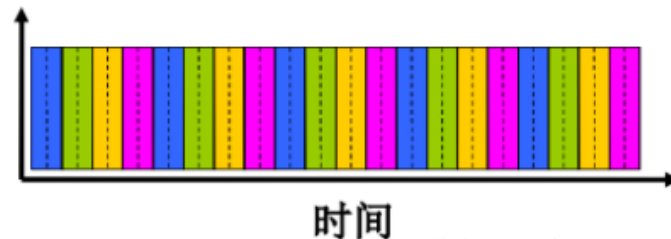
Example:

4 users



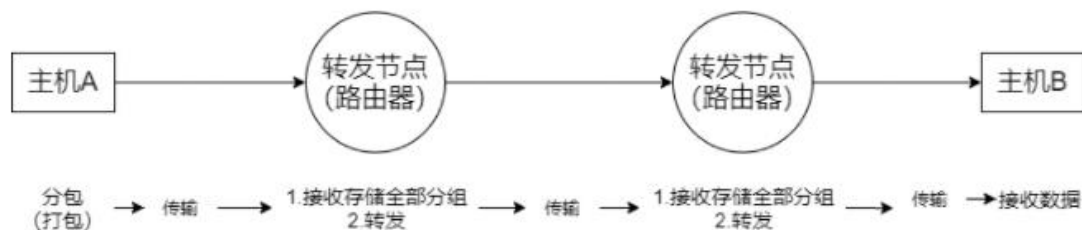
TDM

频率



Network Core : Packet-Switching

包交换网络：将要传输的数据分成一个个的单位：**分组/包 (packet)**。与电路交换网络不同，采用分组交换网路通信时，数据传输会占用通信链路的全部带宽。



主机间的数据传输过程如图：转发节点接收前一个节点传来的分组，并存储起来，等分组的全部传输完成之后，再将数据传输给下一个节点。（存储-转发）

需要注意的是：每一个节点都必须接收分组的全部之后才能向后传递。如果边接收分组，边向后传输，就会造成多段通信链路全部处于占用状态，致使其他用户无法使用。分组也就失去了意义。

Network Core : Circuit-Switching VS Packet-Switching

电路交换网络

原理:

在源结点和目的结点之间建立一条专用的通路用于传送数据，包括建立连接、传输数据和断开连接三个阶段。最典型的电路交换网是传统电话网络。

主要特点:

整个报文的比特流连续地从源点直达终点，好像是在一条管道中传送。用户始终占用端到端的固定传输带宽。

包交换网络（分组交换网络）

原理:

将数据分成较短的固定长度的数据块，在每个数据块中加上目的地址、源地址等辅助信息组成分组（包），以存储-转发方式传输。

主要特点:

是单个分组（整个报文的一部分）传送到相邻结点，存储后查找转发表，转发到下一个结点。

Network Core : Circuit-Switching VS Packet-Switching

电路交换网络

优点：（考题：电路交换的优点是什么）

- 通信时延小
- 实时性强
- 有序传输
- 适用范围广：既适用于传输模拟信号，也适用于传输数字信号
- 控制简单
- 没有冲突

缺点：

- 建立时间长、线路利用率低、灵活性差、不具备差错控制能力（因为不存在路由器的存储并检查数据）。

包交换网络（分组交换网络）

优点：

- 无建立时延。
- 线路利用率高。
- 简化了存储管理(相对于报文交换)。分组的长度固定，相应的缓冲区的大小也固定
- 加速传输。分组是逐个传输的，可以使后一个分组的存储操作与前一个分组的转发操作并行，这种流水线方式减少了报文的传输时间。
- 减少了出错概率和重发数据量。因为分组较短，其出错概率必然减小，所以每次重发的数据量也就大大减少，这样不仅提高了可靠性，也减少了传输时延。

Discussion

- (1) 多路复用解决什么问题？
- (2) 举例说明FDM,TDM的实际应用。

Discussion

- (1) 多路复用解决什么问题？
- (2) 举例说明FDM,TDM的实际应用。

答：

(1) 多路复用解决了电路交换时通信信道被独占，其他信号无法传输，以及资源利用率低的问题。

(2) FDM的实际应用：有线电视；TDM的实际应用：电话系统。

FDM-Frequency Division Multiplexing 频分多路复用

- 将带宽分成几个片段，不同的用户使用不同的片段。

TDM-Time Division Multiplexing 时分多路复用

- 将带宽按周期分为成许多片段，每一个周期再分为几个不同的时隙（slot），每位用户固定使用其中的一个时隙。

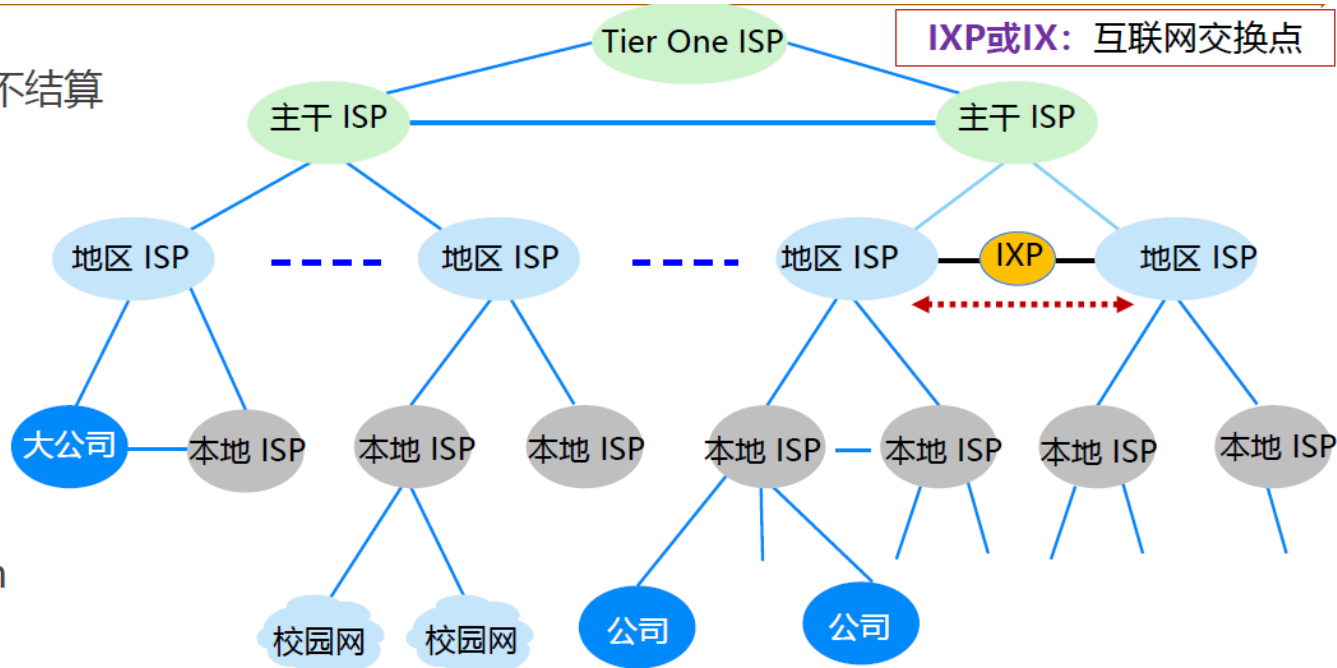
Internet Structure – A Network of Networks

Tier-1 ISP

- 全球最高级别ISP，互不结算
- 中国电信、中国联通
- 美国AT&T
- 美国Verizon
- 美国Sprint
- 日本NTT
- 日本KDDI
- 新加坡SingTel
- 英国British Telecom
- 德国Deutsche Telekom

Tier-2 ISP

- 教育网、中国移动
- 往往需要向更高级别ISP流量费



网络服务提供商 (ISP) 及层次化结构
(实际网络结构要复杂得多)

Performance in Packet-Switched Networks

重要概念1:

- **Transmission Rate 速率**（数据传输速率、数据率、比特率）：指连接到计算机网络上的主机在数字信道上传送数据位数的速率。其单位为b/s（比特/秒，bit/s，有时也写为bps）。最高数据传输速率即为带宽。
 - 在传输领域中（通信），传输速率采用的是十进制， $K=10^3$
 - 在存储领域中，存储容量等使用的是二进制， $K=2^{10}$
- **Bandwidth 带宽**：表示网络的通信线路所能传送数据的能力。通常是指单位时间内从网络中的某一点到另一点所能通过的最高数据传输速率。其单位为b/s。
- **Throughput 吞吐量**：单位时间通过某个网络信道、接口的数据量。其单位为b/s。受带宽、网络额定速率的限制等。
- **速率、带宽、吞吐量的区别与联系，一个形象比喻：**
 - 带宽：一个人数学有考140分的能力
 - 吞吐量：在一次月考中由于肚子疼所以只考了94分
 - 速率：一个学期下来，数学平均分120分



Performance in Packet-Switched Networks

重要概念2:

- Delay 时延**: 指数据（一个报文或分组）从网络（或链路）的一端传送到另一端所需要的总时间，节点时延（Nodal Delay）

由4部分构成：发送时延、传播时延、处理时延、排队时延。总时延=发送时延+传播时延+处理时延+排队时延。

- (1) **Transmission Delay 发送时延**（传输时延）：指数据从网络的一端传输到另一端所需的时间。

- 计算公式为：
$$\text{发送时延} = \frac{\text{数据帧长度 (bit)}}{\text{发送速率 (bit/s)}}$$
 T_{delay}

- (2) **Propagation Delay 传播时延**：信号在传输介质中传播所需的时间。

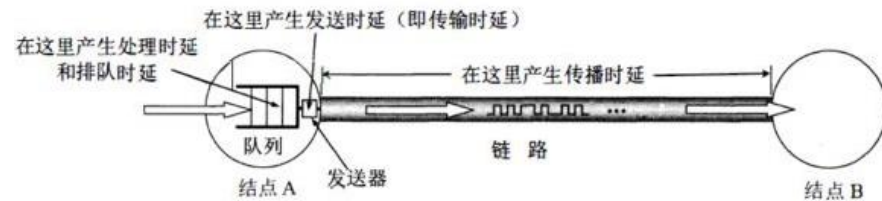
- 计算公式为：
$$\text{传播时延} = \frac{\text{信道长度(m)}}{\text{电磁波在信道上的传播速率(m/s)}}$$
 $T_{\text{propagation}}$

- (3) **Processing Delay 处理时延**：数据在交换节点为存储转发而进行的一些必要的处理所花费的时间。如：分析分组的首部、

从分组中提取数据部分、进行差错检验或查找适当的路由等。计算公式为：处理时延=分组长度/处理速度。 T_{process}

- (4) **Queueing Delay 排队时延**：分组在进入路由器后要先在输入队列中排队等待处理。计算公式为：排队时延=分组长度/

输入速度。



Performance in Packet-Switched Networks

重要概念3:

- **Bandwidth Delay Product 时延带宽积**: 发送端发送的第一个比特即将到达终点时, 发送端已经发出了多少个比特 (以比特为单位的链路长度) 时延带宽积 = 传播时延 × 信道带宽
- **Round Trip Time 往返时延 (RTT)**: 从发送方发送数据开始, 到发送方收到接收方的确认所总共经历的时延
- **Channel Utilization Rate 信道利用率**: 指出某一信道有百分之多少的时间是有数据通过的。 (网络利用率: 全网络的信道利用率的加权平均值)
 - 计算公式: 信道利用率 = 有数据通过时间 / (有数据通过时间 + 无数据通过时间)
- **Traffic Intensity 流量强度**: 指在单位时间内到达系统的请求数与系统处理请求的能力之比。
 - 计算公式: 流量强度 = $\frac{La}{R}$, 其中L指包长度, R指线路带宽, a为平均分组到达率。
 - 平均分组到达率指每秒到达多少包 (packet/s)。
 - 当 $\frac{La}{R} \sim 0$ 时, 平均排队时延就小;
 - 当 $\frac{La}{R}$ 从0逐渐升至1时, 延迟就越来越大;
 - 当 $\frac{La}{R} > 1$ 时, 平均时延趋于无穷, 无限排队, 丢包率上升。

$$\text{Traffic Intensity} = \frac{La}{R}$$

~ 0 平均排队时延 ↓
 $0 \rightarrow 1$ 延迟 ↑
 > 1 平均排队时延 $\rightarrow \infty$
 无限排队 丢包率 ↑

Performance in Packet-Switched Networks

丢包情况的原因: 丢包率 = 丢包数 / 已发分组总数

- 软硬件错误
- 分组到达已满队列，被丢弃

追踪数据包从主机到目的地的路径

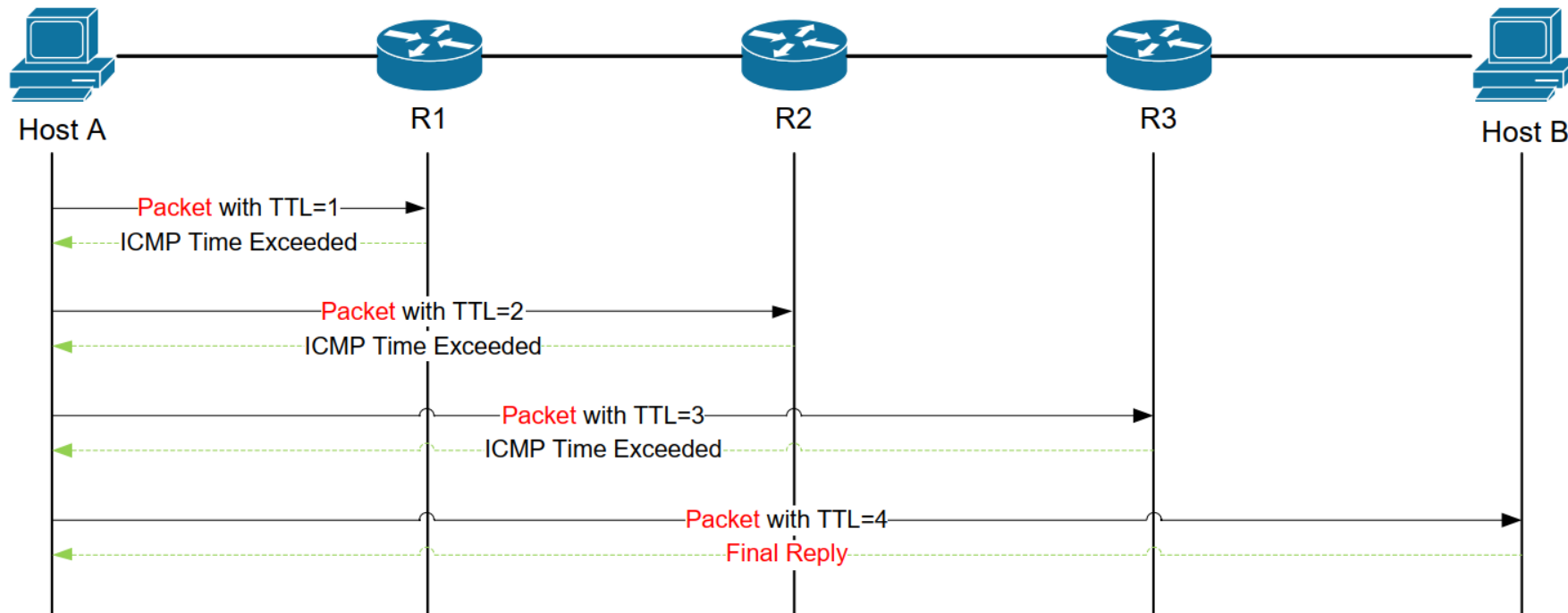
TraceRoute 及其实现原理

计算机网络诊断命令

Traceroute利用了ICMP (Internet Control Message Protocol, 互联网控制报文协议)。这种协议下的报文数据IP头部有一个字段: TTL (Time to Live, 生存时间) 字段。这个字段在最初传输时会被赋值，每经过一个路由器时，TTL减1。当TTL在某一个路由器降为零时，该分组会被抛掉，并向源主机发送一个ICMP报文——通知源主机：分组发送到该路由时被“干掉”了（并附带IP地址），所以实现每一跳的查询方式就是设置不同的TTL。当分组到达目标主机时，肯定会占用一个端口，但是在目标主机中该端口并没有进程在跑，于是目标主机就会向源主机发送另一种ICMP报文——通知源主机：分组在我这边由于目标端口不可达，“挂掉了”（并附带IP地址）。这就是具体的工作原理。

Discussion

TraceRoute的原理?

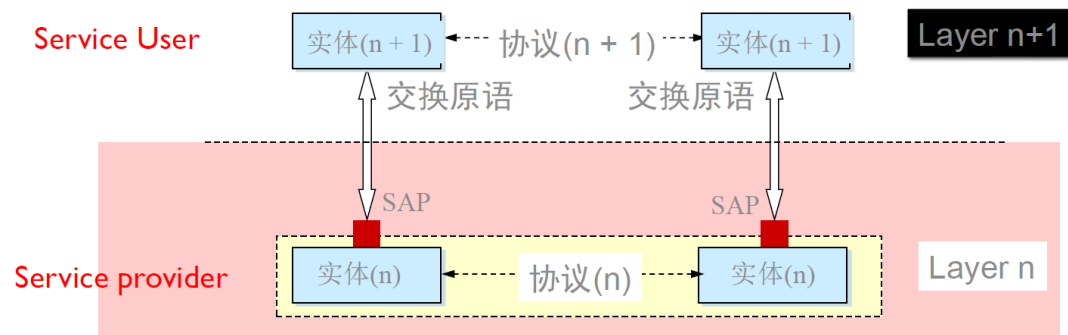


Entity、Protocol、Service and SAP

- **Entity 实体**：表示任何可发送或接收信息的硬件或软件进程。
- **Service 服务**：低层实体向上层实体提供它们之间的通信的能力，是通过原语(primitive)来操作的，是垂直关系。
- **Protocol 协议**：对等层实体(peer entity)之间在相互通信的过程中，需要遵循的规则的组合，是水平关系。
- **Service Access Point 服务访问点 SAP**：上层使用下层提供的服务通过层间的接口。
- **Primitive 原语**：上层使用下层服务的形式，高层使用低层提供的服务，以及低层向高层提供服务都是通过服务访问原语来进行交互的。

协议和服务的关系：

- 协议需要下层提供的服务才能实现，协议实现的目的是为了向上层提供更好的服务。
- 本层的服务用户只能看见服务而无法看见下面的协议，下面的协议对上面的服务用户是透明的（看不见的）。



Layer Functions

层次化的具体思路：

- 将网络复杂的功能分层功能明确的层次，每一层实现了其中一个/一组功能，功能中有其上层可以使用的功能：服务
- 本层协议实体相互交互执行本层的协议动作，目的是实现本层功能，通过接口为上层提供更好的服务
- 在实现本层协议的时候，直接利用了下层所提供的服务
- 本层的服务：借助下层服务实现的本层协议实体之间交互带来的新功能（上层可以利用的）+ 更下层所提供的服务

分层处理和实现复杂系统的好处：

- 概念化：结构清晰，便于标示网络组件，以及描述其相互关系。（分层参考模型）
- 结构化：模块化更易于维护和系统升级。（改变某一层服务的实现不影响系统中的其他层次）

分层处理和实现复杂系统的不好之处：

- 分层实现一个功能时，需要不同层次时时交换信息数据，那么整体的效率就会降低。（每一层都要做差错检测）

Layer Functions

层具有的功能：

- **Error Control 差错控制**：使得两个对等体（不同端系统中相同的层次）之间的逻辑通信更加可靠。
- **Flow Control 流量控制**：防止发送方用PDU淹没接收方。
- **Segment and Reassembly 分段与重组**：在发送端将大数据块分割成小数据块，在接收端将小数据块重新组合成原来的大数据块。
- **Multiplexing 多路复用**：允许多个上层会话共享一个下层连接。
- **Connection Setup 连接建立**：握手。

Internet Protocol Stack

Protocol Data Unit 协议数据单元 PDU：是指在计算机网络中，数据在不同层之间传输时所使用的数据单元。每个层次都将数据添加到PDU中，以便在下一层中使用。PDU通常由头部和有效载荷组成。

应用层 (application layer)：完成应用报文之间的交互。（实现各种网络应用）

- 数据单元：报文 (message)
- eg: FTP、SMTP、HTTP

传输层 (transport layer)：实现进程到进程之间的数据传输。

- 数据单元：报文段 (segment)
- eg: TCP、UDP

网络层 (network layer)：实现端到端（源主机-目标主机）的以分组为单位的数据传输。

- 数据单元：分组 (packet)；如果是无连接的方式：数据报 (datagram)
- eg: IP、routing protocols (路由协议)

链路层 (link layer)：在相邻网络节点间以帧为单位传输数据。

- 数据单元：帧 (frame)
- eg: PPP (点对点协议)、802.11(wifi)、Ethernet

物理层 (physical layer)：在线路上传送bit。（把数字数据转换成物理信号，使其承载于媒体之上）

- 数据单元：位 (bit)

Internet Protocol Stack

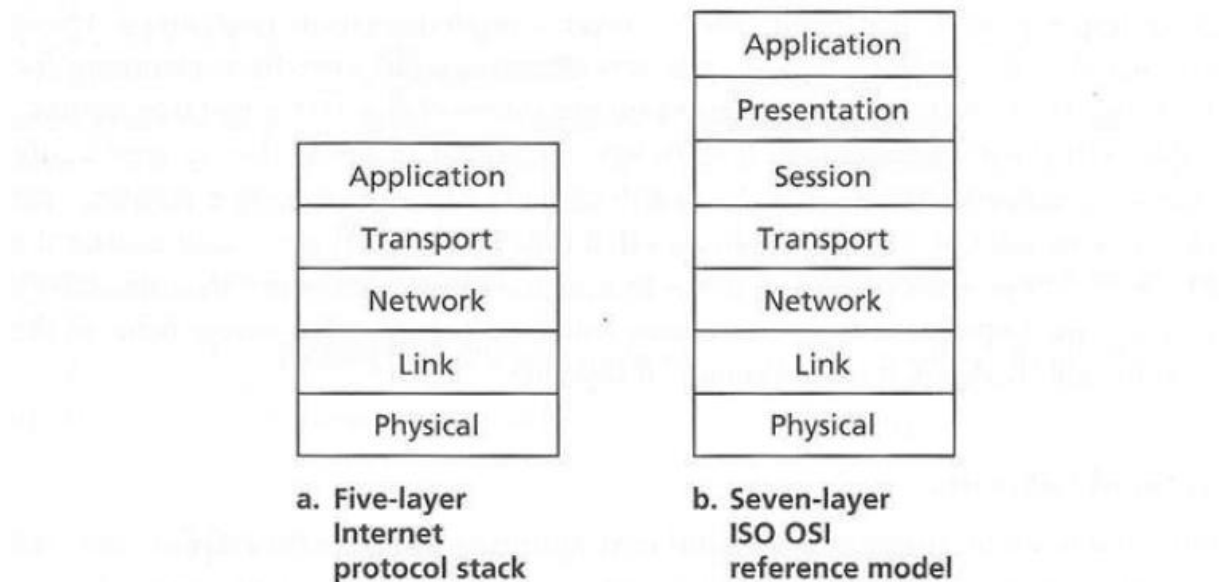


Figure 1.23 ♦ The Internet protocol stack (a) and OSI reference model (b)

Services Layering & Encapsulation

封装与解封装

源主机向目标主机传送报文的真正流程——不断地封装与解封：

上层报文的传输需要借助层间接口，依托于下层的服务，在不同层加入相应的控制信息，形成本层的数据单元，继续向下传递，直到物理层将每一个bit传送给通信链路（这是封装的过程）。通过链路交换机，底部物理层会还原数据形成链路层的数据单元（帧），链路层查询帧头部的目标MAC地址，并查询该交换机的栈表（或交换表），并决定将帧从哪一个端口放出，接着就交给该端口的网卡，转换成对应物理层的bit，将其释放到通信链路中。通过路由器，先向上传递形成帧，再将帧转换为网络层的数据单元（分组），网络层从分组获取目标的IP信息，并查询网络层的转发表，决定将数据从哪一个网口释放出去，再形成对应的帧向下传递，进而转为bit由物理层将其传输到通信链路。就这样数据不断传输，最后到达目标主机，并不断解封装形成应用层的报文。

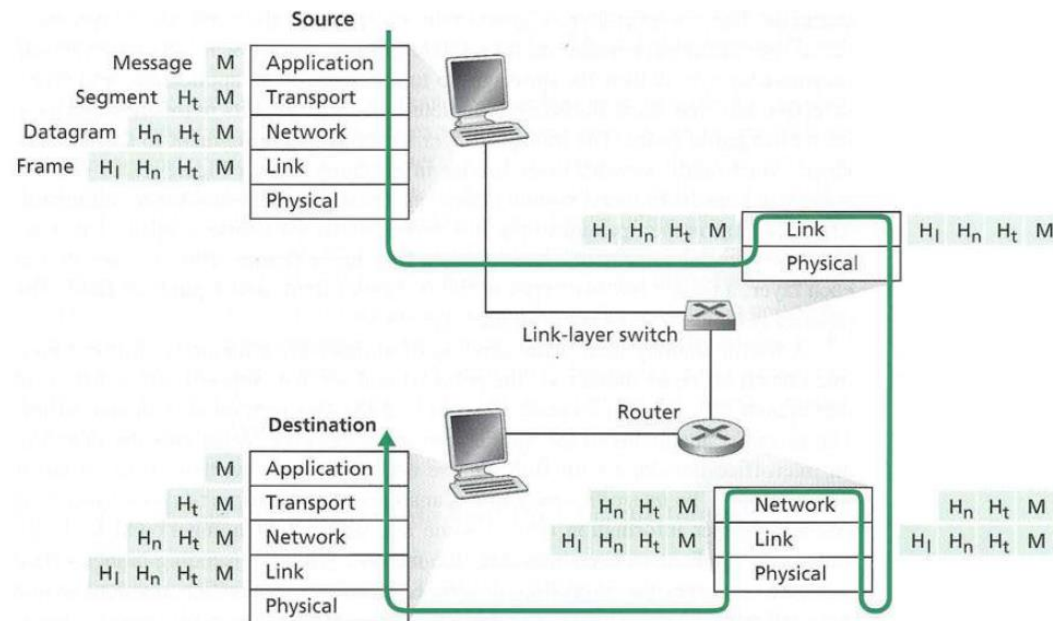
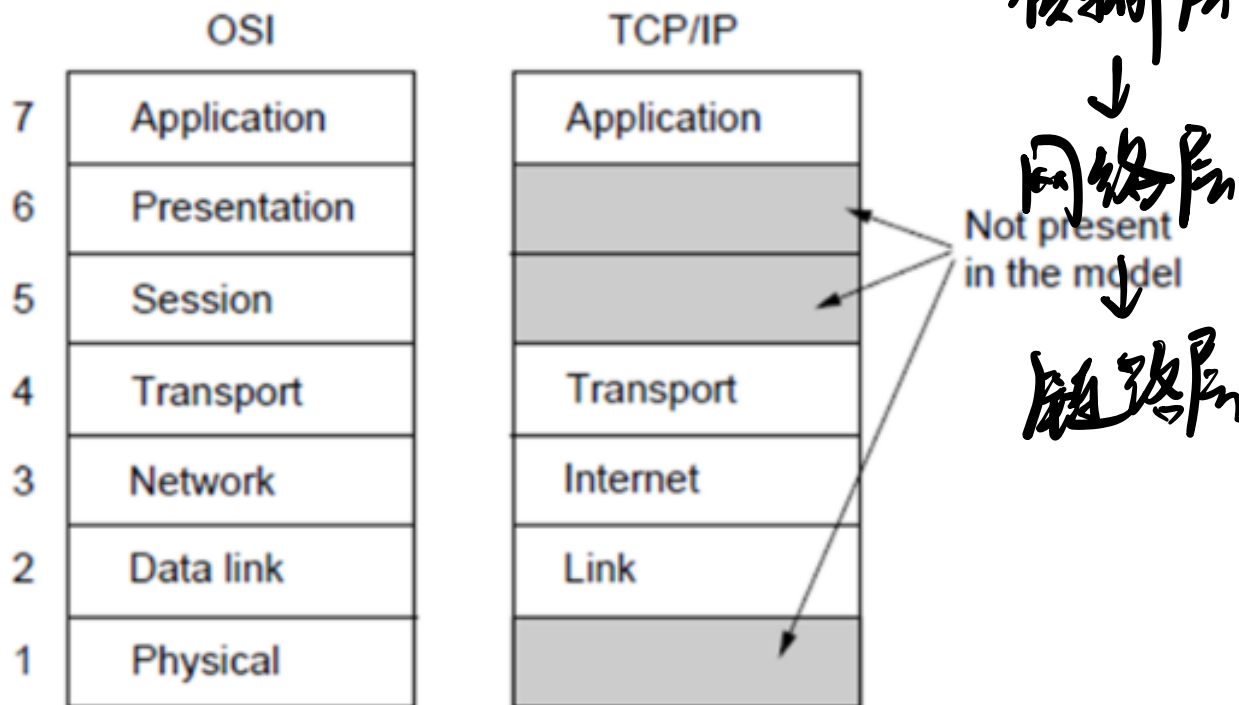


Figure 1.24 ♦ Hosts, routers, and link-layer switches; each contains a different set of layers, reflecting their differences in functionality

Discussions

OSI & TCP/IP



Application Layer

Network applications are the *raisons d'etre* of a computer network - if we couldn't conceive of any useful applications, there wouldn't be any need for networking infrastructure and protocols to support them.

Network Application Architectures

Client-Server(C/S)

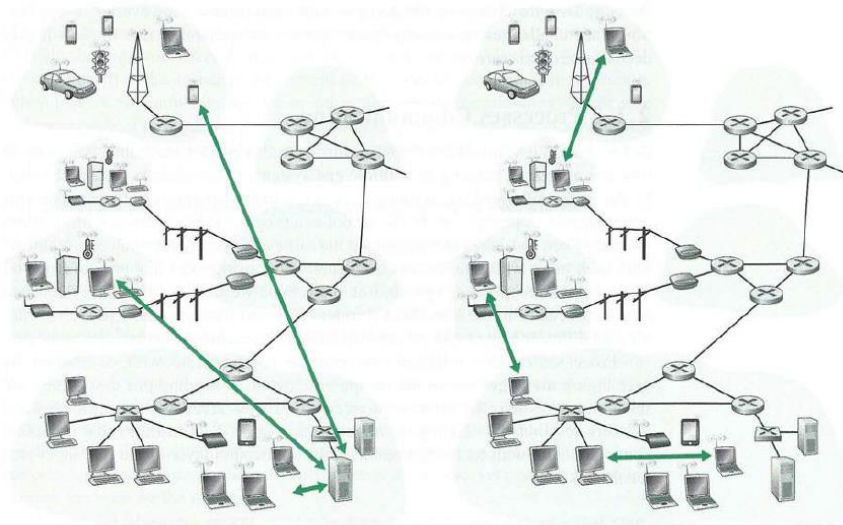
- 服务器-客户端架构。客户是服务请求方，服务器是服务提供方。客户必须知道服务器的地址，而反之则不必。
- 客户端主动与服务器连接，服务器存在固定IP地址和周知的端口号，而客户可以是动态的IP地址。
- 服务器“一直在运行”。

Peer-to-peer(P2P)

- 每一个节点既是客户端又是服务器，任意端系统之间可以进行通信。
- (几乎)没有一直运行的“服务器”。
- 自扩展性：新peer节点带来新的服务能力，当然也带来新的服务请求
- 参与的主机间歇性连接且可以改变IP地址

Hybrid

- 如Skype、QQ。



a. Client-server architecture

b. Peer-to-peer architecture

Figure 2.2 • (a) Client-server architecture; (b) P2P architecture

Many of today's most popular and traffic-intensive applications are based on P2P architectures. These applications include file sharing (e.g., BitTorrent), peer-assisted download acceleration (e.g., Xunlei) and Internet telephony and video conference (e.g., Skype). The P2P architecture is illustrated in Figure 2.2(b). We mention that some applications have hybrid architectures, combining both client-server and P2P elements. For example, for many instant messaging applications, servers are used to track the IP addresses of users, but user-to-user messages are sent directly between user hosts (without passing through intermediate servers).

Processes Communicating

Client&Server Processes

- Client Process 客户进程：发起通信的一方
- Server Process 服务器进程：等待连接的一方

Socket

- 进程通过套接字接收和发送信息
- 可以把套接字比作一道门。发送进程将报文推出门户，发送进程依赖于传输层设施在另外一侧的门将报文交付给接收进程，同样的，接收进程从另外一端的门户收到报文(依赖于传输层设施)

Addressing Processes

- 进程为了接收报文，必须有一个标识，即SAP。
- IP地址+端口号可标示进程。
- 主机IP：唯一的32位IP地址(IPv4);
- 端口号：如HTTP的80端口。

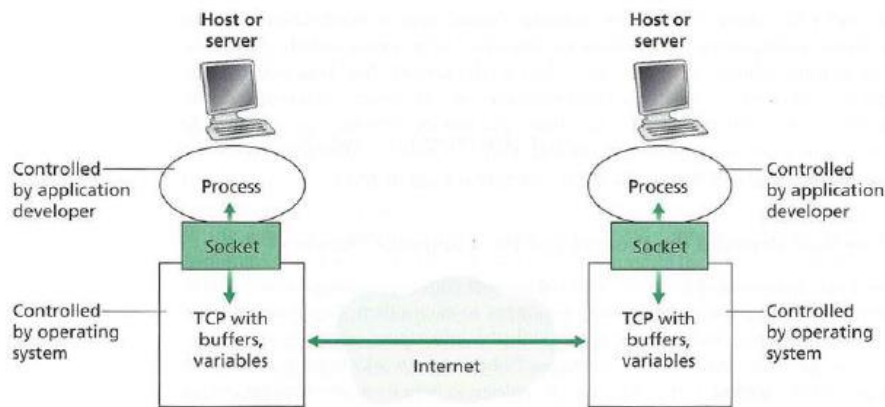


Figure 2.3 Application processes, sockets, and underlying transport protocol

Transport Services Available To Applications



TCP (Transmission Control Protocol)

可提供:

- Reliable data transfer 可靠数据传输
- Flow control 流量控制
- Congestion control 拥塞控制

不提供:

- Timing 延时
- Minimum throughput guarantee 最小带宽保证
- Security 安全性

特点:

- Connection-oriented: TCP为面向连接的协议, 需要在客户和服务端之间建立连接

How to secure TCP?

- ✓ SSL(Secure Socket Layer)
- ✓ TLS(Transport Layer Security)

UDP (User Datagram Protocol)

不提供:

- Reliability 可靠传输
- Flow control 流量控制
- Congestion control 拥塞控制
- Timing 延时
- Throughput guarantee 带宽保证
- Security 安全性
- Connection setup 建立连接

特点:

- Unreliable data transfer: 不可靠数据传输



Why bother to have UDP?

- ✓ IP协议中并没有端口(port)的概念
- ✓ UDP相较TCP有延迟小、数据传输效率高, 开销更小的优势, 其简单、传输快、可一对多或者多对多。
- ✓ 对于延迟敏感的应用, 少量的数据丢失一般可以被忽略, 这时使用UDP传输将能够提升用户的体验。

Transport Services Available To Applications

SSL/TLS

保护TCP安全性

SSL/TLS 过程

- 1."client hello"消息:** 客户端通过发送"client hello"消息向服务器发起握手请求, 该消息包含了客户端所支持的 TLS 版本和密码组合以供服务器进行选择, 还有一个"client random"随机字符串。
- 2."server hello"消息:** 服务器发送"server hello"消息对客户端进行回应, 该消息包含了数字证书, 服务器选择的密码组合和"server random"随机字符串。
- 3.验证:** 客户端对服务器发来的证书进行验证, 确保对方的合法身份, 验证过程可以细化为以下几个步骤:
 1. 检查数字签名
 2. 验证证书链 (这个概念下面会进行说明)
 3. 检查证书的有效期
 4. 检查证书的撤回状态 (撤回代表证书已失效)
- 4."premaster secret"字符串:** 客户端向服务器发送另一个随机字符串 "premaster secret (预主密钥)", 这个字符串是经过服务器的公钥加密过的, 只有对应的私钥才能解密。
- 5.使用私钥:** 服务器使用私钥解密 "premaster secret"。
- 6.生成共享密钥:** 客户端和服务器均使用 client random, server random 和 premaster secret, 并通过相同的算法生成相同的共享密钥 KEY。
- 7.客户端就绪:** 客户端发送经过共享密钥 KEY 加密过的 "finished" 信号。
- 8.服务器就绪:** 服务器发送经过共享密钥 KEY 加密过的 "finished" 信号。
- 9.达成安全通信:** 握手完成, 双方使用对称加密进行安全通信。

SSL / TLS 握手过程



Socket

Transport Services Available To Applications

TCP Socket - TCP套接字

- 连接对象个数：仅支持一对一

TCP套接字通讯

- 4元组：源IP，源port，目标IP，目标port
- 唯一的指定了一个会话(2个进程之间的会话关系)
- 应用使用这个标示，与远程的应用进程通信
- ✓ 不必在每一个报文的发送都要指定这4元组
- 简单，便于管理

UDP Socket – UDP套接字

- 连接对象个数：支持一对一、一对多、多对一、多对多

UDP套接字通讯

- 2元组：目标IP，目标port
- UDP套接字指定了应用所在的一个端节点(end point)
- 在发送数据报时，采用创建好的本地套接字(标识 ID)，就不必在发送每个报文中指明自己所采用的 ip和port
- ✓ 但是在发送报文时，必须要指定对方的ip和udp port(另外一个端节点)

Transport Services Available To Applications

应用需要传输层提供的服务：

1. Data loss 数据丢失率

有些应用则要求100%的可靠数据传输(如文件)，有些应用(如音频)能容忍一定比例以下的数据丢失

2. Timing 延迟

一些应用出于有效性考虑，对数据传输有严格的时间限制(如：Internet 电话、交互式游戏)

3. Throughput 吞吐

一些应用(如多媒体应用)必须需要最小限度的吞吐，从而使得应用能够有效运转。一些应用能充分利用可供使用的吞吐(“弹性应用”)

4. Security 安全性

机密性、数据完整性、可认证性(鉴别).... “CIA”

Application Layer Protocols

应用层协议		依赖传输层协议
Application	Application-Layer Protocol	Underlying Transport Protocol
Electronic mail	SMTP [RFC 5321]	TCP
Remote terminal access	Telnet [RFC 854]	TCP
Web	HTTP [RFC 2616]	TCP
File transfer	FTP [RFC 959]	TCP
Streaming multimedia	HTTP (e.g., YouTube)	TCP
Internet telephony	SIP [RFC 3261], RTP [RFC 3550], or proprietary (e.g., Skype)	UDP or TCP

Figure 2.5 ♦ Popular Internet applications, their application-layer protocols, and their underlying transport protocols

Application	Application-Layer Protocol	Underlying Transport Protocol
Electronic mail	SMTP	TCP
Remote terminal access	Telnet	TCP
Web	HTTP	TCP
File transfer	FTP	TCP
Remote file server	NFS	Typically UDP
Streaming multimedia	typically proprietary	UDP or TCP
Internet telephony	typically proprietary	UDP or TCP
Network management	SNMP	Typically UDP
Name translation	DNS	Typically UDP

Figure 3.6 ♦ Popular Internet applications and their underlying transport protocols

补充:

- SSH(Secure Shell)是一种建立在应用层基础上的安全协议，其为C/S架构，传输层协议基于TCP。
- DNS为C/S架构，传输层协议基于UDP。

Application Ports

协议/服务名称	端口号	简介
ftp	20、21	File Transfer Protocol 文件传输协议，20用于连接，21用于传输
ssh	22	Secure Shell 安全外壳协议，专为 远程登录 会话和其他网络服务提供安全性的协议
http	80	Hyper Text Transfer Protocol 超文本传输协议，用于网页浏览
DNS	53	Domain Name System 域名系统，域名解析
https	443	Hypertext Transfer Protocol Secure 超文本传输安全协议，用于安全浏览网页
www代理服务	8080	Apache Tomcat web server，进行网页浏览
smtp	25	Simple Mail Transfer Protocol 简单邮件传输协议
telnet	23	不安全的文本传送
pop3	110	Post Office Protocol

可在互联网上搜到很多类似的端口总结，如
<https://zhuanlan.zhihu.com/p/77322667>
<https://www.cnblogs.com/forforever/p/13224320.html>

Discussions

Classify each of the scenarios below as client-server, P2P, or hybrid, and explain your answer briefly. Answering these questions may require some Web surfing.

- a. TaoBao/JD
- b. QQ
- c. BitTorrent
- d. Telnet
- e. DNS

Discussions

Classify each of the scenarios below as client-server, P2P, or hybrid, and explain your answer briefly. Answering these questions may require some Web surfing.

a. TaoBao/JD

a.C/S 京东与淘宝均有自己固定的服务器，为用户提供登录购买等服务，用户彼此间不直接建立通信

b. QQ

b.hybrid 登录注册时，用户是以C/S形式向中心服务器存入数据。在具体通信时，用户间通过P2P，（但用户需要先通过中心服务器获得好友的IP地址）

c. BitTorrent

c.P2P 进行文件下载时，用户之间相互转发彼此拥有的文件部分，并没有传统的下载服务器等。

d. Telnet

d.C/S telnet实现远程登录时，需要客户向服务器发出请求

e. DNS

e.C/S DNS是域名解析系统，由客户向域名服务器发出数据请求，实现IP与域名互相转换

HTTP Overview

HTTP 特点

- Stateless 无状态的协议(服务器不会保留客户端过去发出的请求)
- 如何记住用户? / 如何保存用户状态? Cookies
- 使用TCP连接
- HTTP常用端口号: 80
- HTTPS常用端口号: 443

None Persistent HTTP / Persistent HTTP

- A HTML webpage with 10 jpegs

`http://www.someSchool.edu/someDepartment/home.index`

Here is what happens:

1. The HTTP client process initiates a TCP connection to the server `www.someSchool.edu` on port number 80, which is the default port number for HTTP. Associated with the TCP connection, there will be a socket at the client and a socket at the server.
2. The HTTP client sends an HTTP request message to the server via its socket. The request message includes the path name `/someDepartment/home.index`. (We will discuss HTTP messages in some detail below.)
3. The HTTP server process receives the request message via its socket, retrieves the object `/someDepartment/home.index` from its storage (RAM or disk), encapsulates the object in an HTTP response message, and sends the response message to the client via its socket.
4. The HTTP server process tells TCP to close the TCP connection. (But TCP doesn't actually terminate the connection until it knows for sure that the client has received the response message intact.)
5. The HTTP client receives the response message. The TCP connection terminates. The message indicates that the encapsulated object is an HTML file. The client extracts the file from the response message, examines the HTML file, and finds references to the 10 JPEG objects.
6. The first four steps are then repeated for each of the referenced JPEG objects.

1. HTTP 客户端向服务器发送 TCP 请求
客户端与服务器会建立连接套接字

2. HTTP 客户端通过套接字向服务器发送 HTTP 请求

3. HTTP 服务器通过套接字接收请求信息，索引并检索请求中的响应信息，然后通过套接字将响应信息发送回客户端

4. HTTP 服务器告诉 TCP 关闭 TCP 连接

5. HTTP 客户端接收响应信息，TCP 连接终止

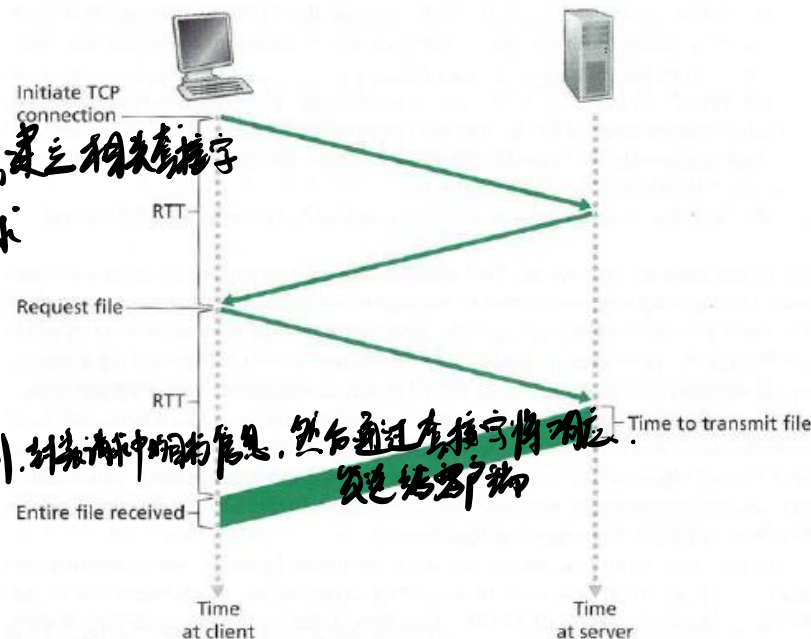


Figure 2.7 Back-of-the-envelope calculation for the time needed to request and receive an HTML file

None-Persistent HTTP – Without Pipelining

11 connections generated

$$T = 11 * (2RTT + T_{trans})$$

Establish TCP connection: 2RTT

None Persistent HTTP / Persistent HTTP

None-Persistent HTTP – Pipelining

$$T = 2RTT(\text{Establish Connection}) + 2 * RTT(10\text{jpeg-Parallel})$$

Persistent HTTP – Without Pipelining

$$T = 2RTT(\text{Establish Connection}) + 10 * RTT(10\text{jpeg})$$

Persistent HTTP – Pipelining

$$T = 2RTT(\text{Establish Connection}) + RTT(10\text{jpeg-Parallel})$$

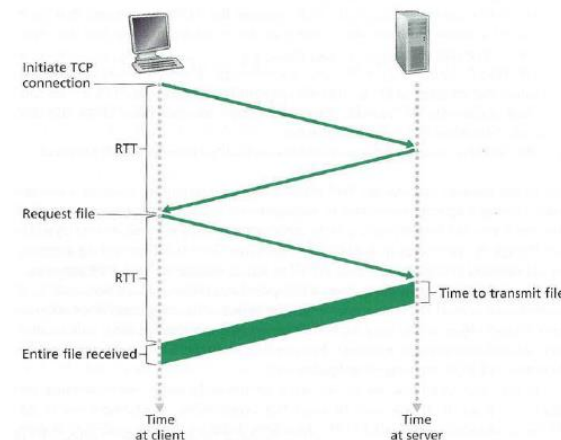


Figure 2.7 • Back-of-the-envelope calculation for the time needed to request and receive an HTML file

响应时间: 一个RTT用来发起TCP连接+一个 RTT用来HTTP请求并等待HTTP响应+文件传输时间 $2RTT + T_{trans}$

非持久HTTP的缺点: 请求每个对象需要2个 RTT; 操作系统必须为每个TCP连接分配资源; 浏览器通常打开并行TCP连接, 以获取引用对象

持久HTTP连接: 服务器在发送响应后, 仍保持TCP连接; 在相同客户端和服务端之间的后续请求和响应报文通过相同的连接进行传送; 客户端在遇到一个引用对象的时候, 就可以尽快发送该对象的请求

非流水方式的持久HTTP: 客户端只能在收到前一个响应后才能发出新的请求; 每个引用对象需要一个RTT

流水方式的持久HTTP: HTTP/1.1的默认模式; 客户端遇到一个引用对象就立即产生一个请求; 所有引用 (小) 对象基本上只用一个RTT时间就能满足

HTTP request message

提交表单有两种方式:

- **POST方式:**
 - 网页通常包括表单输入
 - 输入的数据放在实体(entity body)部分上传到服务器
- **URL方式:**
 - 方法: GET方法
 - 输入的数据放在URL中上传

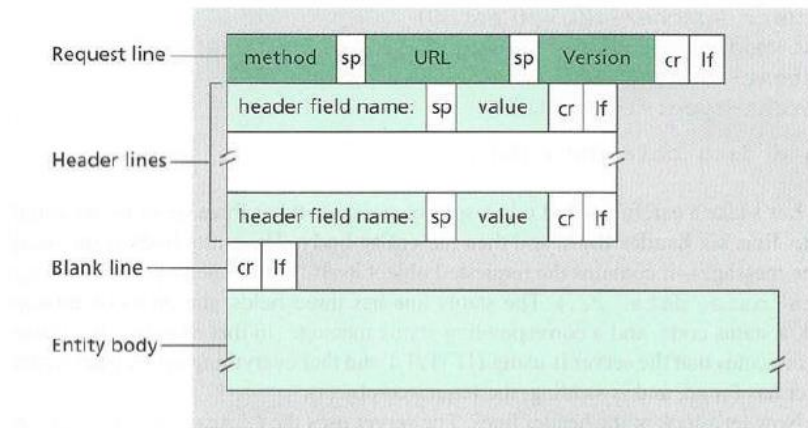


Figure 2.8 ♦ General format of an HTTP request message



```

request line (GET, POST, HEAD commands) → GET /index.html HTTP/1.1\r\n
header lines → Host: www.someschool.edu\r\n
  User-Agent: Firefox/3.6.10\r\n
  Accept: text/html,application/xhtml+xml\r\n
  Accept-Language: en-us,en;q=0.5\r\n
  Accept-Encoding: gzip,deflate\r\n
  Accept-Charset: ISO-8859-1,utf-8;q=0.7\r\n
  Keep-Alive: 115\r\n
  Connection: keep-alive\r\n
carriage return, line feed at start of line indicates end of header lines → \r\n
carriage return character → \r
line-feed character → \n
  
```

HTTP response message

HTTP响应状态码:

- 200 OK: 请求成功, 请求对象包含在响应报文的后续部分。
- 301 Moved Permanently: 请求的对象已经被永久转移了; 新的URL在响应报文的Location: 首部行中指定。客户端软件自动用新的URL去获取对象。
- 400 Bad Request: 一个通用的差错代码, 表示该请求不能被服务器解读。
- 404 Not Found: 请求的文档在该服务上没有找到。
- 505 HTTP Version Not Supported

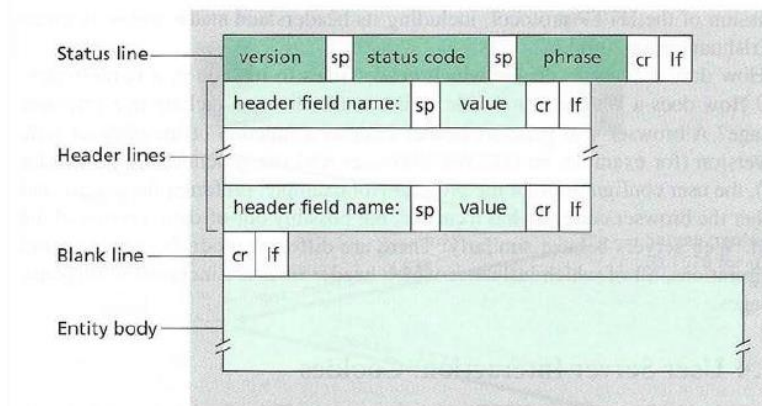


Figure 2.9 ♦ General format of an HTTP response message

```

status line
(protocol
status code
status phrase) → HTTP/1.1 200 OK\r\n
Date: Sun, 26 Sep 2010 20:09:20 GMT\r\n
Server: Apache/2.0.52 (CentOS)\r\n
Last-Modified: Tue, 30 Oct 2007 17:00:02 GMT\r\n
ETag: "17dc6-a5c-bf716880"\r\n
Accept-Ranges: bytes\r\n
Content-Length: 2652\r\n
Keep-Alive: timeout=10, max=100\r\n
Connection: Keep-Alive\r\n
Content-Type: text/html; charset=ISO-8859-1\r\n
\r\n
header lines
data, e.g., requested HTML file → data data data data data ...
  
```

Cookies: User-Server Interaction

cookies有4个组成部分:

- 在HTTP响应报文中有一个cookie的首部行
- 在HTTP请求报文含有一个cookie的首部行
- 在用户端系统中保留有一个cookie文件, 由用户的浏览器管理
- 在Web站点有一个后端数据库

cookies可以带来以下内容:

- authorization 授权 (用户验证)
- shopping carts 购物车
- recommendations 推荐
- user session state (Web e-mail) 用户会话状态

cookies的副作用:

- Cookies允许站点知道许多关于用户的信息 (隐私)
- 网站记录用户信息并分析用户行为 (可能将它知道的东西卖给第三方)
- 使用重定向和cookie的搜索引擎还能知道用户更多的信息 (姓名、电话、邮箱等)

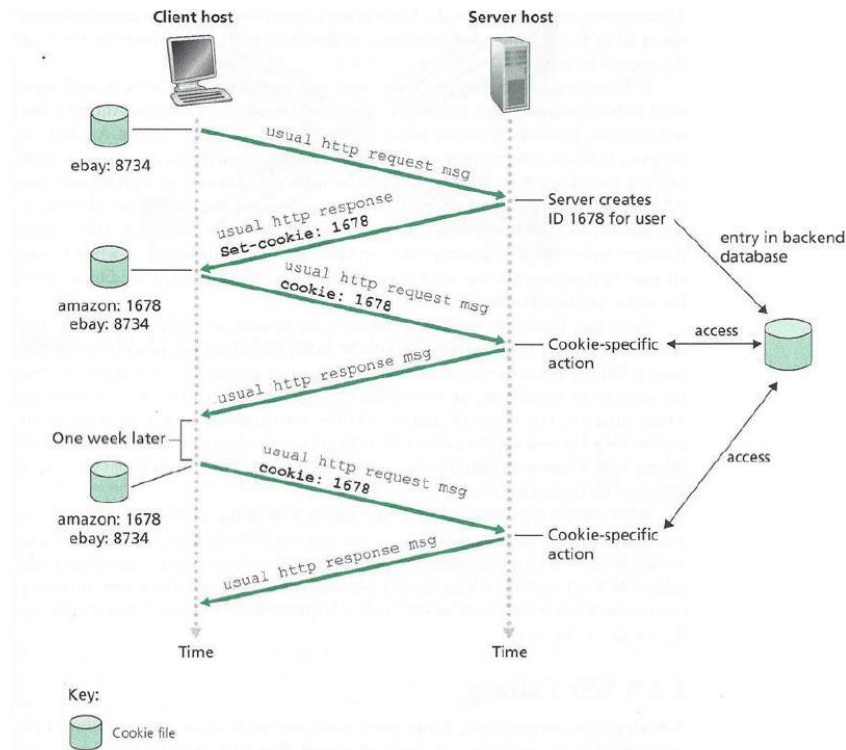


Figure 2.10 ♦ Keeping user state with cookies

Web Caching / Proxy Server

目标：不访问原始服务器，就可以满足客户的请求。

浏览器将所有的HTTP请求发给缓存（代理服务器）

- 在缓存中的对象：缓存直接返回对象
- 如对象不在缓存，缓存请求原始服务器，然后再将对象返回给客户端

优势：

- 降低客户端的请求响应时间
- 可以大大减少一个机构内部网络与Internet接入链路上的流量
- 互联网大量采用了缓存：可以使较弱的ICP也能够有效提供内容（快速分发内容）

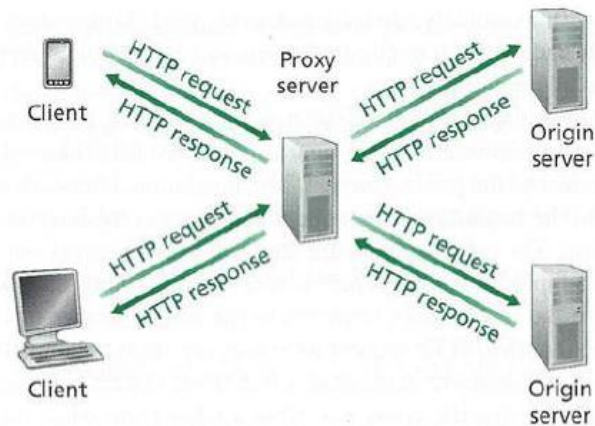


Figure 2.11 ♦ Clients requesting objects through a Web cache

Web Caching-Conditional GET

使用缓存有一个**风险**：客户端访问到的缓存中的数据，在原始服务器中已修改，结果就是拿到旧数据。

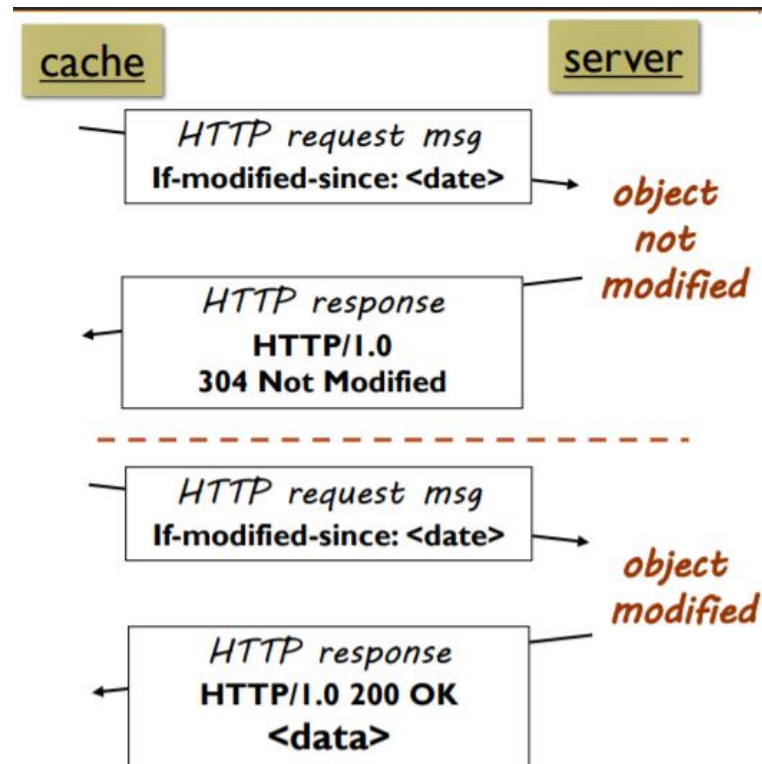
为此，Conditional GET的**目标**：如果缓存器中的对象拷贝是最新的，就不要发送对象。

➤ cache: specify date of cached copy in HTTP request

If-modified-since: <date>

➤ server: response contains no object if cached copy is up-to-date:

HTTP/1.0 304 Not Modified



Discussions

使用COOKIE技术有什么好处？COOKIE通常会收集什么信息？从隐私保护角度看，有什么问题？

Discussions

使用COOKIE技术有什么好处？COOKIE通常会收集什么信息？从隐私保护角度看，有什么问题？

(1) 好处：Cookie在一定程度上将HTTP从无状态协议转变为有状态协议。通过Cookie，服务器可以保持状态和上下文，从而实现有状态的交互。

(2) 一般会收集：

用户身份验证信息：如用户ID、会话ID等，用于识别和验证用户身份。

用户偏好设置：如语言选择、主题颜色等用户个性化设置。

浏览历史：用户访问过的页面、点击的链接等。

购物车内容：电子商务网站中用户添加到购物车的商品信息。

跟踪信息：用于广告和分析目的的用户行为数据，如点击次数、浏览时间等。

(3) 存在隐私侵犯、数据泄露、用户追踪、数据滥用、XSS/CSRF偷Cookie等。

Electronic Mail: Email

互联网邮件系统三组件:

- user agents (用户代理)
- mail servers (邮件服务器)
- simple mail transfer protocol: SMTP (简单邮件传输协议)

SMTP: Simple Mail Transfer Protocol

- 使用TCP在客户端和服务端之间传送报文, 端口号为25;
- 直接传输: 从发送方服务器到接收方服务器
- 传输的3个阶段: 握手、传输报文、关闭
- 命令/响应交互:
 - 命令: ASCII文本
 - 响应: 状态码和状态信息
- 报文必须为7位ASCII码 (邮件的内容必须是7位ASCII码的范围, 超过是不允许传输的) - 可通过MIME解决

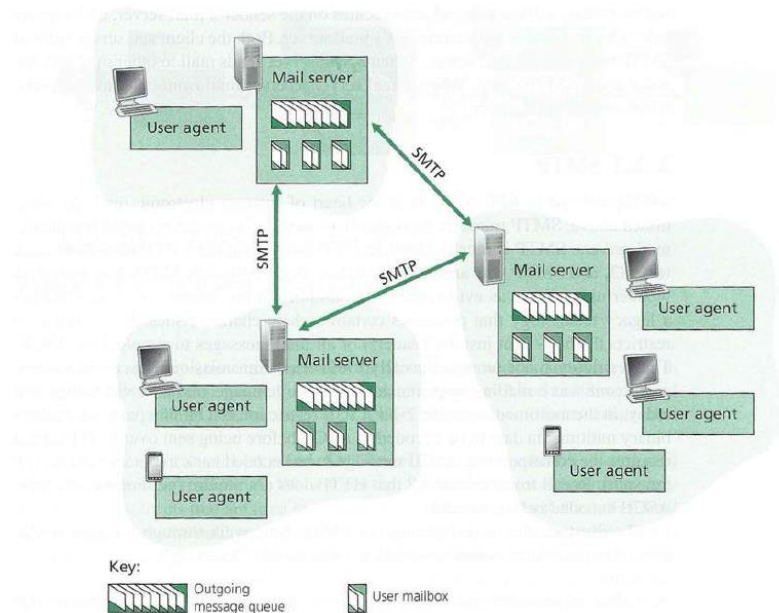


Figure 2.14 ♦ A high-level view of the Internet e-mail system

```

S: 220 hamburger.edu
C: HELO crepes.fr
S: 250 Hello crepes.fr, pleased to meet you
C: MAIL FROM: <alice@crepes.fr>
S: 250 alice@crepes.fr... Sender ok
C: RCPT TO: <bob@hamburger.edu>
S: 250 bob@hamburger.edu ... Recipient ok
C: DATA
S: 354 Enter mail, end with "." on a line by itself
C: Do you like ketchup?
C: How about pickles?
C: .
S: 250 Message accepted for delivery
C: QUIT
S: 221 hamburger.edu closing connection
  
```


Email Transferring & Mail Message Formats

问题：如果传输的内容不在ASCII范围之内？

解答：对其进行编码（扩展）：

- **MIME：多媒体邮件扩展**（Multimedia Mail Extension），RFC 2045, 2056
- 在报文首部用额外的行申明MIME内容类型
- 内部的编码采用base64编码格式，将不在ASCII范围之内内容进行扩展，使得其在ASCII范围之内。（大小写英文字母，加号，等号）

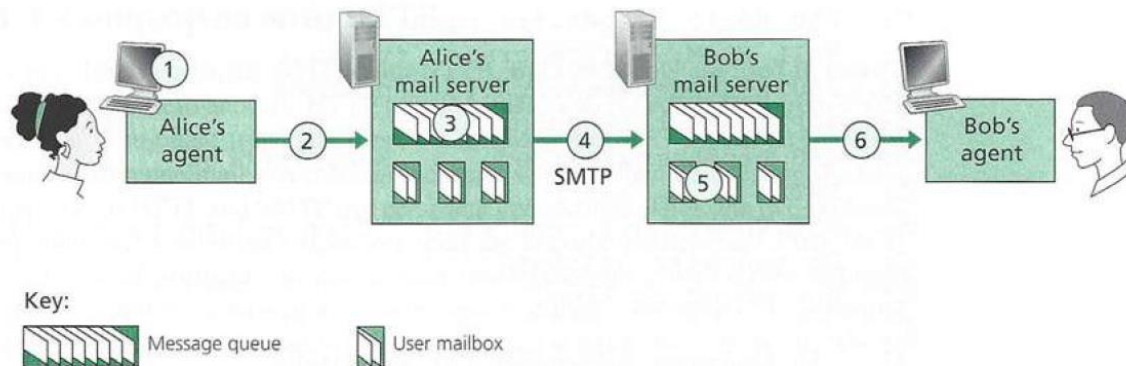


Figure 2.15 ♦ Alice sends a message to Bob

1. Alice使用用户代理撰写邮件并发送给**bob@some school.edu**
2. Alice的用户代理将邮件发送到她的邮件服务器；邮件放在报文队列中（SMTP）
3. SMTP的客户端打开到Bob邮件服务器的TCP连接 **建立TCP连接**
4. SMTP客户端通过TCP连接发送Alice的邮件（SMTP） **通过TCP发送邮件**
5. Bob的邮件服务器将邮件放到Bob的邮箱
6. Bob调用他的用户代理阅读邮件（POP3、IMAP、HTTP）

SMTP VS HTTP

SMTP

- SMTP: 推 (push) 协议
- SMTP使用持久连接
- SMTP要求报文（首部和主体）为7位ASCII编码
- SMTP服务器使用 CRLF, CRLF决定报文的尾部
- SMTP: 多个对象包含在一个报文中

HTTP

- HTTP: 拉 (pull) 协议
- 二者都是ASCII形式的命令/响应交互、状态码
- HTTP: 每个对象封装在各自的响应报文中

Mail Access Protocols

- 邮件访问协议：从服务器访问邮件（“拉下来”）
- **POP**：邮局访问协议（Post Office Protocol）[RFC 1939]
 - 用户身份确认 (代理<-->服务器) 并下载
- **IMAP**：Internet邮件访问协议（Internet Mail Access Protocol）[RFC 1730]
 - 相比POP3更多特性 (更复杂，如：远程目录维护；仅仅只下载邮件正文，附件可选择下载，对手机端友好)
 - 直接在邮件服务器上处理存储的报文
- **HTTP**：Hotmail, Yahoo! Mail等 (Webmail)
 - 方便

Discussions

电子邮件系统使用TCP传送邮件，为什么有时会遇到邮件发送失败的情况？为什么有时对方会收不到发送的邮件？

Discussions

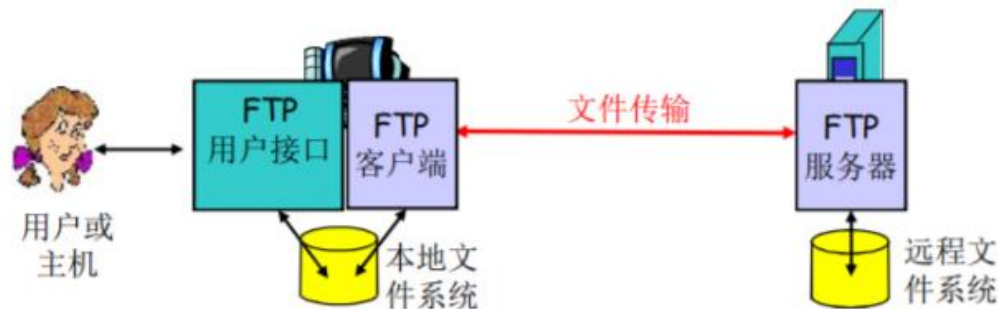
电子邮件系统使用TCP传送邮件，为什么有时会遇到邮件发送失败的情况？为什么有时对方会收不到发送的邮件？

1. 收件人地址错误
2. SMTP服务器故障
3. 电子邮件过大
4. 被识别为垃圾邮件被过滤
5. 邮件服务器队列溢出

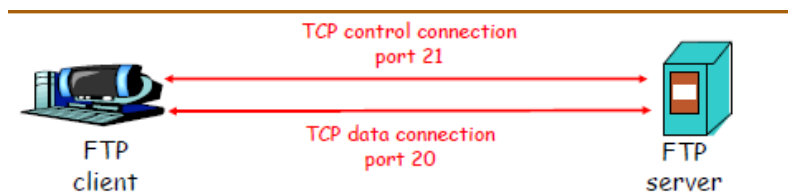
FTP Overview

FTP特点

- 向远程主机上传文件或从远程主机接收文件
- 建立在TCP基础上，控制端口为21，数据传输端口为20。
- C/S模式，“带外传输”



FTP Control & Data Connection



FTP传输过程

控制端口

1. FTP客户端与FTP服务器通过端口21联系，并使用TCP为传输协议
2. 客户端通过控制连接获得身份确认（通过TCP）（用户名和口令，**全部都为明文**）
3. 客户端通过控制连接发送命令浏览远程目录（可以上传、下载）
4. 收到一个文件传输命令时，服务器打开一个到客户端的TCP数据传输连接（客户端的20号端口，由服务器主动建立），控制信息/指令和数据传输的连接是不一样的。
5. 一个文件传输完成后，服务器关闭连接
6. 服务器打开第二个TCP数据连接用来传输另一个文件
7. 控制连接：带外（“out of band”）传送（只能建立一个）
8. 数据连接：带内传送（可以建立多个）
9. FTP服务器维护用户的状态信息：当前路径、用户帐户与控制连接对应（FTP是一个有状态的协议）

建立一个控制连接 → 带外传送
建立多个数据连接 → 带内传送

DNS Overview

- 主要思路：
 - 分层的、基于域的命名机制
 - 若干分布式的数据库完成名字到IP地址的转换
 - 运行在UDP之上端口号为53的应用服务
 - 核心的Internet功能，但在端系统（边缘）中的应用层实现
- 主要目标：
 - 实现主机名-IP地址的转换(name/IP translate)
 - 其他目的：
 - 主机别名-规范名字的转换：Host aliasing
- 规范名字便于管理，主机别名便于访问
 - 邮件服务器别名-邮件服务器的正规名字的转换：Mail server aliasing
 - 负载均衡：Load Distribution

C/S 架构

UDP 端口 53

应用层实现

主机名 $\longleftrightarrow$ IP地址

Why not Centralize DNS?

不集中化 DNS ?

DNS采用了Client-Server的方式了，为啥不把DNS集中化呢？这是一个书中提到的问题，不把DNS作为集中设计主要是考虑到以下几点：

- 第一点是单点故障问题 (Single-point Failure)。当DNS集中到一点时，这一点的服务挂了，那么整个互联网就没法用域名服务了。这点很好理解。
- 第二点就是流量问题 (Traffic Volume)。2022年全世界的数据总量大概有84.5ZB，也就是8.45乘以10的22次方这么多字节。想想如果全世界的DNS查询包都要前往一个服务器的样子。流量太大，导致没法集中化。
- 第三点是集中化数据库带来的传输问题 (Distant Centralized Database)。想象把集中的DNS服务器放在成都，那么如果一个美国人想访问美国的网站，他的查询就会跑半圈地球，再绕半圈地球回去。这导致的拥塞和延迟是很可怕的。
- 最后就是维护问题 (Maintenance)。这一个集中的DNS数据库就得存世界上所有的IP地址和域名的映射，那么当每一个域名的映射更新的时候，都需要更新一下数据库，而且该数据库的总量将会是大到非常恐怖的。

总而言之，它无法衡量！ Doesn't scale!

Distributed、Hierarchical Database & TLD/Authoritative DNS Servers

仅从一个层面命名设备会有很多重名，因而DNS采用层次树状结构的命名方法：

- Internet根被划为几百个顶级域(top level domains, TLD)
 - 通用 (generic)：如.com .edu .gov
 - 国家的 (country)：如.cn .uk .sn
- 每个(子)域下面可划分为若干子域(subdomains)
- 树叶是主机

Root DNS Servers

- 共13个根域名服务器

TLD Servers

- 负责顶级域名和所有国家级的顶级域名

Authoritative DNS Servers

- 组织自己的DNS服务器，提供IP到域名的权威映射

Local DNS Servers

- 每个ISP提供的服务器（又名default DNS server）
- 存在本地的DNS缓存
- 如代理服务器一样工作

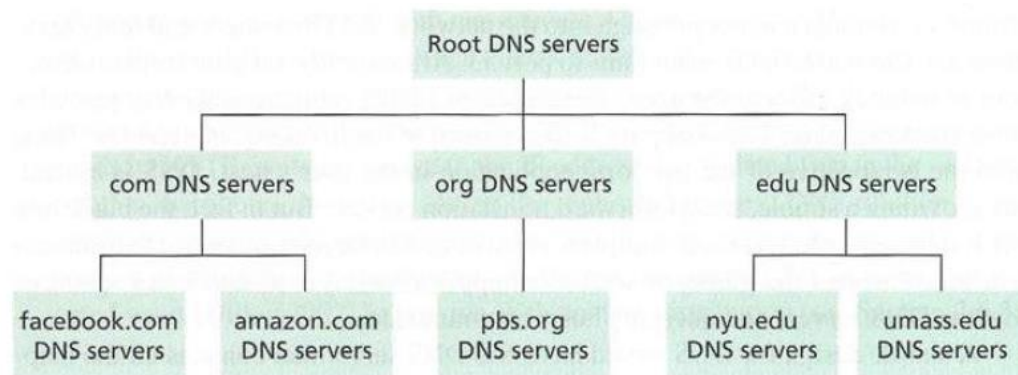


Figure 2.17 ♦ Portion of the hierarchy of DNS servers

DNS Name Resolution Process

递归查询的问题：名字解析负担都放在当前联络的名字服务器上，根服务器的负担太重

为此出现了迭代查询

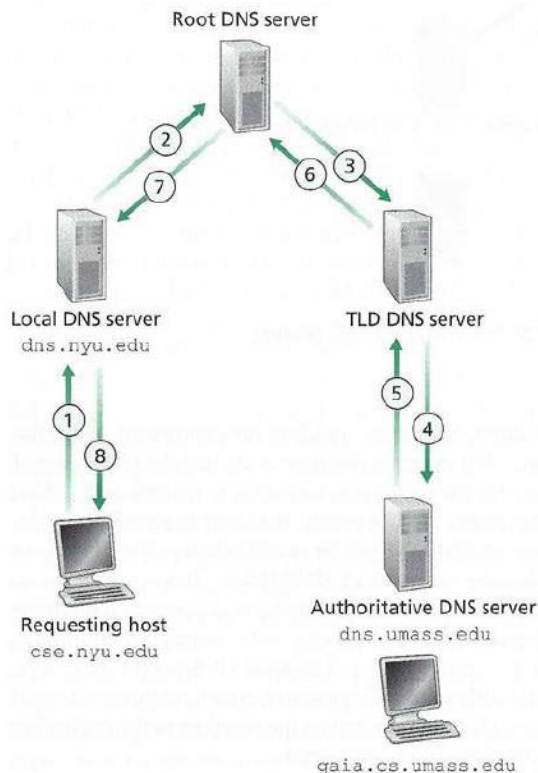


Figure 2.20 ♦ Recursive queries in DNS

Recursive DNS Query

递归查询

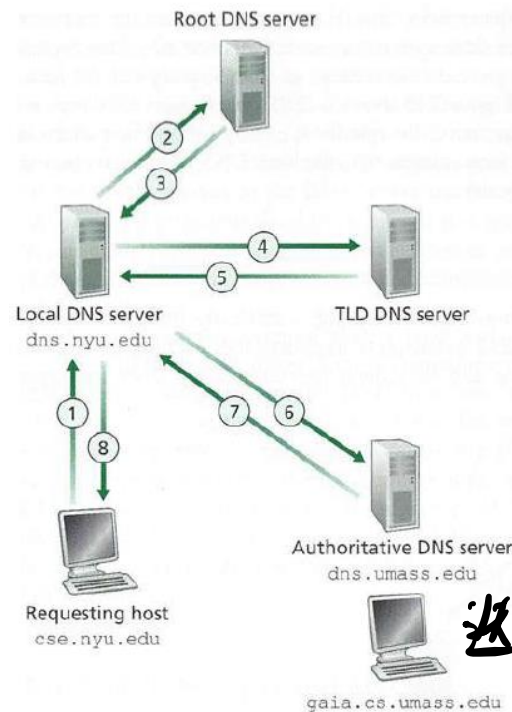


Figure 2.19 ♦ Interaction of the various DNS servers

Iterative DNS Query

迭代查询

DNS Records & DNS Messages

DNS记录

RR格式: (domain_name, ttl, type, class, Value)

Domain_name: 域名

TTL: Time To Live, 生存时间 (权威、缓冲记录)

Type: 类别: 资源记录的类型

- Type=**A**: Name为主机、Value为IP地址
- Type=**CNAME**: Name为规范名字的别名 (如: www.ibm.com 的规范名字为servereast.backup2.ibm.com) 、value 为规范名字
- Type=**NS**: Name为域名(如foo.com)、Value为该域名的权威服务器的域名
- Type=**MX**: Value为name对应的邮件服务器的名字

Class: 对于Internet, 值为IN

Value: 可以是数字, 域名或ASCII串

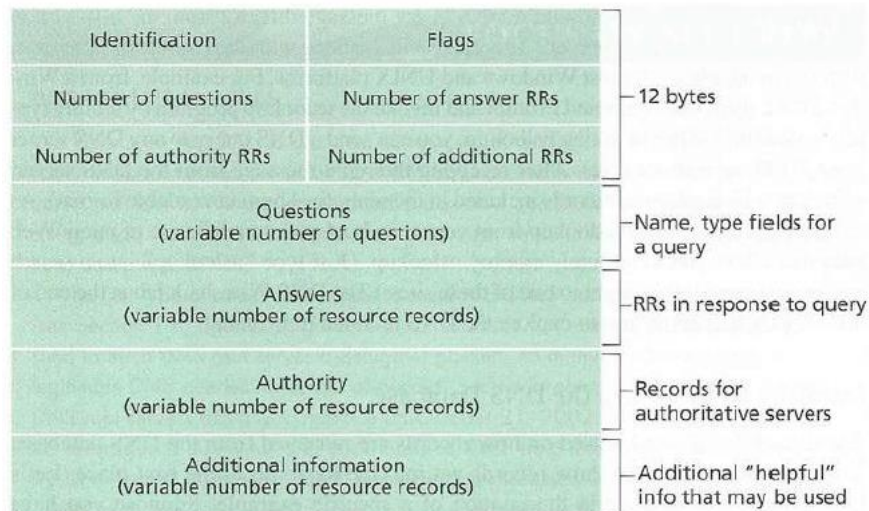


Figure 2.21 ♦ DNS message format

DNS Cache

提高DNS性能：DNS缓存

- 一旦名字服务器学到了一个映射，就将该映射缓存起来
- 根服务器通常都在本地服务器中缓存着（使得根服务器不用经常被访问）
- 目的：提高效率
- 可能存在的问题：如果情况变化，缓存结果和权威资源记录不一致
- 解决方案：TTL（默认2天）

Discussions

美国屏蔽.cn域名会让我们上不了网吗？如果美国断开对中国的DNS根服务器会怎么样？

Discussions

✓ 美国屏蔽.cn域名会让我们上不了网吗？如果美国断开对中国的DNS根服务器会怎么样？

在主根被篡改后，当其他根服务器缓存过期后，全球所有人都无法访问.cn后缀的网站。但我国维护着根镜像，所以我国控制着根镜像中的内容，所以可以不同步修改，甚至可以直接搭一个主根。

国内用户想访问.cn后缀的网站，会落到对根镜像的访问上；若其他国家想访问.cn后缀的网站，他们会把.cn记录加回去，并拒绝同步美国删去的这几行。DNS根域名服务器无法被同时全部关闭，包括根子服务器。

随着我国IPv6的全面推进、部署覆盖和根服务器的建立，IPv4系统中的DNS根服务器关闭对我国互联网的影响会越来越小。

P2P File Transfer

假设文件大小为 F , u_s 为服务器上传速率, d_i 是节点下载速率, u_i 是节点上传速率

文件分发时间: C/S模式

- 服务器传输: 都是由服务器发送给peer, 服务器必须顺序传输 (上载) N 个文件拷贝:
 - 发送一个copy: F/u_s
 - 发送 N 个copy: NF/u_s
- 客户端: 每个客户端必须下载一个文件拷贝
 - $d_{\min}$ = 客户端最小的下载速率
- 下载带宽最小的客户端下载的时间: $F/d_{\min}$

采用C/S方法将一个 F 大小的文件分发给 N 个客户端耗时:

$$D_{C/S} \geq \max(NF/u_s, F/d_{\min})$$

- 当客户端数量很少时, 影响传输时间的瓶颈是客户端的下载速率;
- 当客户端数量很多时, 影响传输时间的瓶颈是服务器端的上载速率。

文件分发时间: P2P模式

- 服务器传输: 最少需要上载一份拷贝
 - 发送一个拷贝的时间: F/u_s
- 客户端: 每个客户端必须下载一个拷贝
 - 最小下载带宽客户单耗时: $F/d_{\min}$
- 客户端: 所有客户端总体下载量 NF
 - 最大上载带宽是: $u_s + \sum u_i$ 时间: $\frac{F}{u_s + \sum u_i}$
- 除了服务器可以上载, 其他所有的peer节点都可以上载

采用P2P方法 将一个 F 大小的文件分发给 N 个客户端耗时:

$$D_{P2P} \geq \max(F/u_s, F/d_{\min}, NF/(u_s + \sum u_i))$$

- 随着客户端的数量增多, P2P模式的优势就体现出来了: 客户端数量越多, 较C/S模式节约的时间就越多。

客户端数量多 P2P

P2P File Transfer

文件分发时间：

C/S模式 vs P2P模式

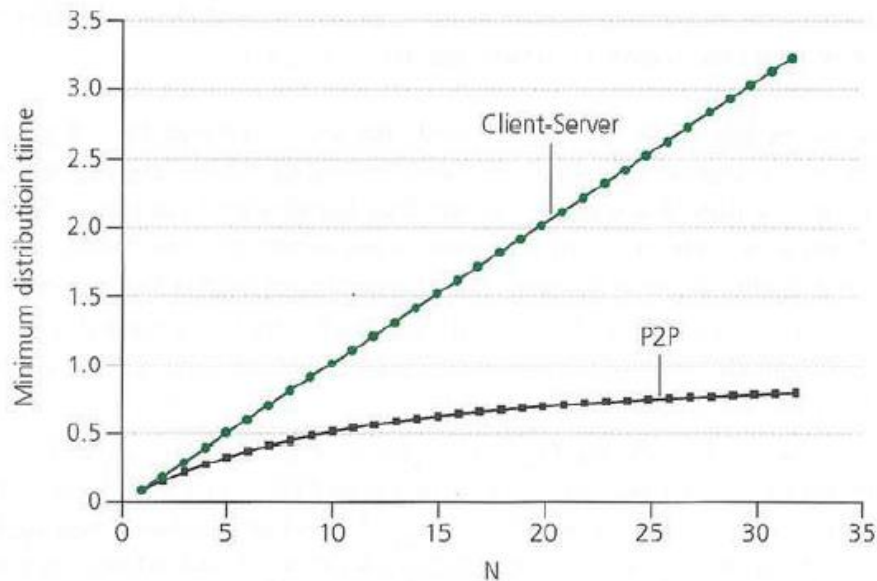


Figure 2.23 ♦ Distribution time for P2P and client-server architectures

P2P File Transfer

文件分发时间：

C/S模式 vs P2P模式

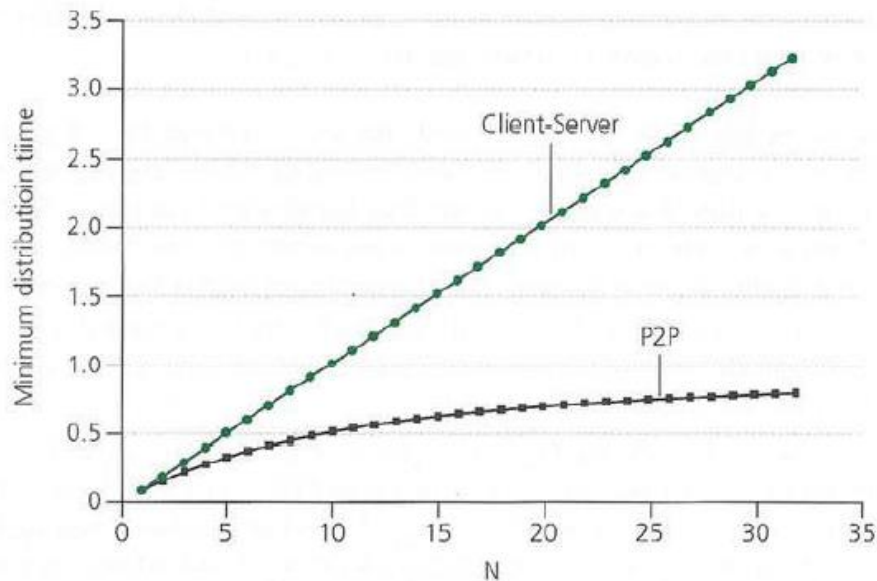


Figure 2.23 ♦ Distribution time for P2P and client-server architectures

Bittorrent

其余概念：

- 文件被分为一个个块256KB
- 网络中的这些peers发送接收文件块，相互服务
- **tracker**: 跟踪torrent中参与节点
- Torrent（洪流）：节点的组，之间交换文件块

当Alice加入网络中：

Alice加入到网络中，首先需要从跟踪服务器处获得peer节点列表， 然后开始和torrent中的peer节点交换块。

Peer如何加入torrent：

- 一开始没有块，但是将会通过其他节点处累积文件块
- 向跟踪服务器注册，获得peer节点列表，和部分peer节点构成邻居关系（“连接”）
- 在任何给定时间，不同peer节点拥有一个文件块的子集
- **周期性的**， Alice节点向邻居询问他们拥有哪些块的信息
- Alice向peer节点请求它希望的块， **稀缺的块优先**

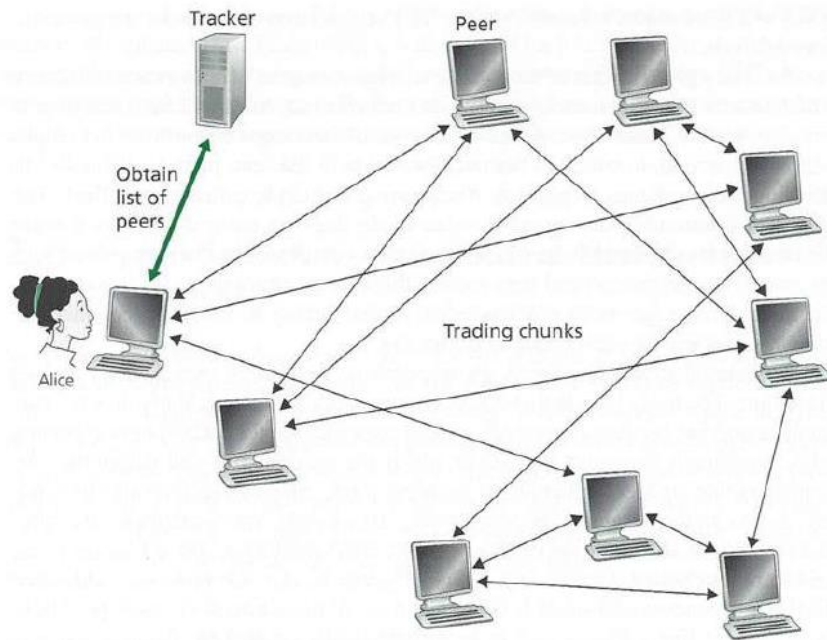


Figure 2.24 • File distribution with BitTorrent

Bittorrent

当Alice加入网络中： 一报还一报

发送块（一报还一报 tit-for-tat）

Alice向4个peer发送块， 这些块向它自己提供最大带宽的服务

- 其他peer被Alice阻塞 (将不会从Alice处获得服务)
- 每10秒重新评估一次： 前4位 (top-4 providers)
- 每个30秒： 随机选择其他peer节点， 向这个节点发送块
- “优化疏通” 这个节点
- 新选择的节点可以加入这个top 4

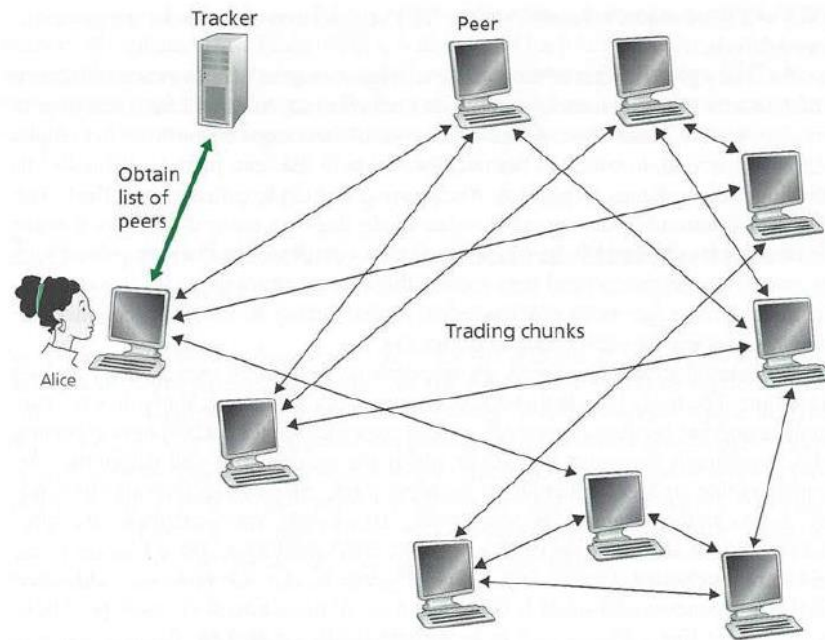


Figure 2.24 ♦ File distribution with BitTorrent

Discussions

BT下载、PT下载与版权

PT (Private Tracker) 和BT (BitTorrent) 在技术上共享同一个基础协议，即BitTorrent协议。BitTorrent是一种P2P (Peer-to-Peer) 文件共享协议，允许用户分块下载文件并同时上传已下载的部分，形成一个去中心化的文件共享网络。PT站是在BitTorrent协议基础上，通过私人追踪器 (Private Tracker) 对文件共享进行更严格的管理和控制。

PT由于实施会员制，故相较BT，其一般拥有更高的资源质量、更快的下载质量、更严格的访问控制和分享率要求。

由于开放性和去中心化，P2P下载更难追踪和管理用户行为，这使得版权侵权问题更为普遍。用户分享大量未经授权的内容，仍然存在严重的版权侵权问题。

CDN – Video Content Delivery as an Example

1. The user visits the Web page at NetCinema.
2. When the user clicks on the link `http://video.netcinema.com/6Y7B23V`, the user's host sends a DNS query for `video.netcinema.com`.
3. The user's Local DNS Server (LDNS) relays the DNS query to an authoritative DNS server for NetCinema, which observes the string "video" in the host-name `video.netcinema.com`. To "hand over" the DNS query to KingCDN, instead of returning an IP address, the NetCinema authoritative DNS server returns to the LDNS a hostname in the KingCDN's domain, for example, `a1105.kingcdn.com`.
4. From this point on, the DNS query enters into KingCDN's private DNS infrastructure. The user's LDNS then sends a second query, now for `a1105.kingcdn.com`, and KingCDN's DNS system eventually returns the IP addresses of a KingCDN content server to the LDNS. It is thus here, within the KingCDN's DNS system, that the CDN server from which the client will receive its content is specified.
5. The LDNS forwards the IP address of the content-serving CDN node to the user's host.
6. Once the client receives the IP address for a KingCDN content server, it establishes a direct TCP connection with the server at that IP address and issues an HTTP GET request for the video. If DASH is used, the server will first send to the client a manifest file with a list of URLs, one for each version of the video, and the client will dynamically select chunks from the different versions.

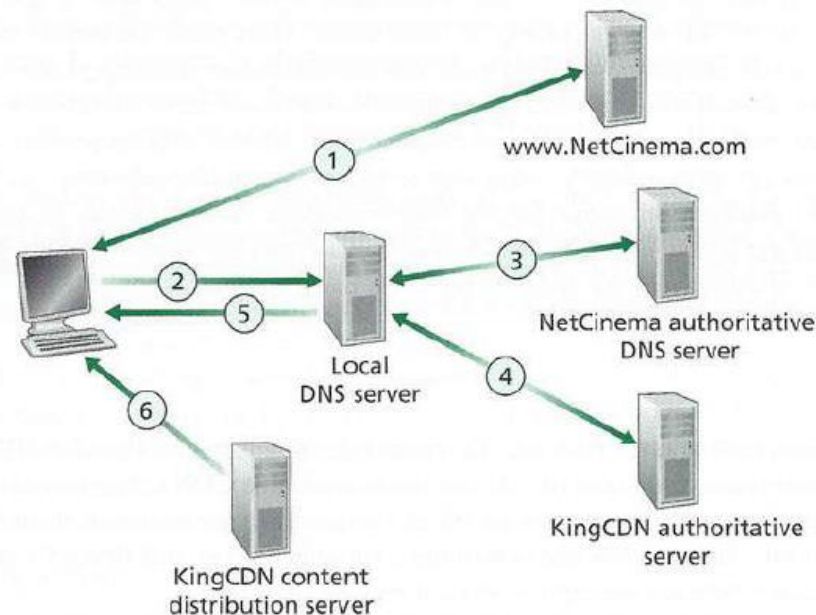


Figure 2.25 ♦ DNS redirects a user's request to a CDN server

绕过CDN寻找真实网站IP的一些方法
<https://zhuanlan.zhihu.com/p/33440472>

计网复习

第一部分

本PPT只是概述！
本PPT可能存在错误！

如果本PPT出现和教材/PPT/老师所讲的知识存在
出入，那么请以教材/PPT/老师所讲的知识为准！